Overview on clustering algorithms and closed-loop learning

Cerioli Gabriele

Supervisor: Prof. Matteo Marsili

October 2025

1 Clustering problems

Clustering problems involve grouping data into clusters based on similarities, used in unsupervised machine learning.
Basically clustering is finding a structure in a collection of unlabeled data.

Often the similarity criterion is distance: distance-based clustering.

The goals of Clustering can be different:

- finding representatives for homogenous groups (DATA REDUCTION)

- finding natural clusters and describe their unknown properties
- finding unusual data objects (OUTLIER DETECTION)

The main requirements for a clustering algorithm are:

- dealing with different types of attributes and with clusters of arbitrary shape
- dealing with noise and outliers
- high dimensionality (working with high-dimension dataset)

1.1 Distance measure

An important component of a clustering algorithm is the distance measure between data points. If different data vectors have same physical units we can use the Euclidean distance metric.

But there is a problem: Euclidean distance can sometimes be misleading, different scalings can lead to different clusterings:

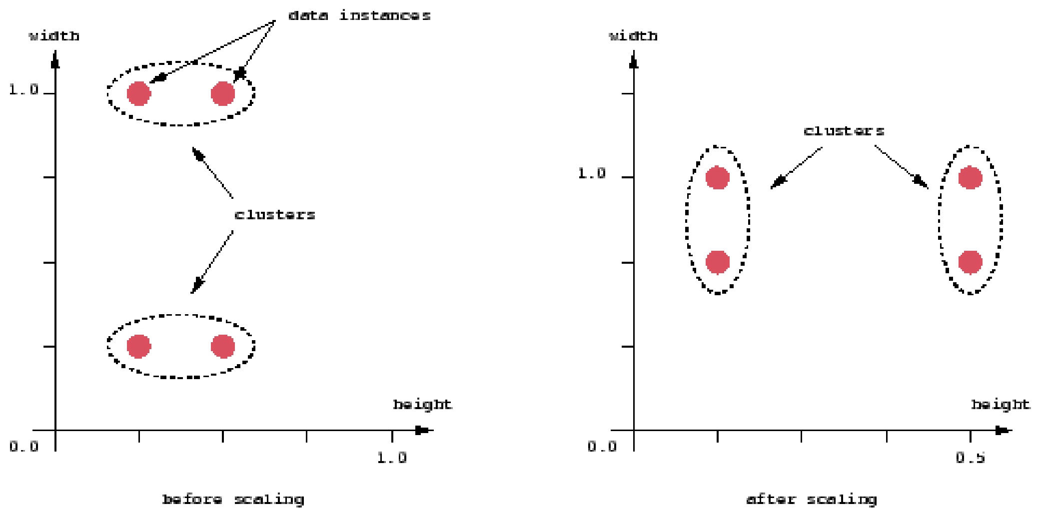

Figure 1: Different scaling can lead to different clustering results if we use the Euclidean distance

2 Hierarchical clustering

An hierarchical clustering algorithm is based on the union between the two nearest clusters. The initial condition is a single cluster, and then the algorithm after a few iterations reaches the final clusters wanted.

3 Partitional clustering

3.1 K-Means clustering

K-means (MacQueen, 1967) is one of the simplies clustering algorithms. It classifies a given dataset through a certain number of clusters (k) fixed a priori.

4 Spectral clustering

5 Closed-loop learning clustering

6 Evaluating the performance of clustering algorithms

6.1 Metrics

6.1.1 Purity score

$$\text{Purity} = \frac{1}{N}\sum_{k=1}^{K}\max_j |\omega_k \cap c_j| \tag{1}$$

where ω_k is the set of elements of (predicted) cluster k, c_j is the set of elements of (real) class j, and N is the total number of elements.

If we can calculate the confusion matrix then we find that $|\omega_k \cap c_j|$ are the max elements in each coloumn.
Then we can find the purity score simply:

```
import numpy as np
from sklearn.metrics import confusion_matrix
contingency_matrix = confusion_matrix(y_true, y_pred)
max_per_cluster = np.max(contingency_matrix, axis=0)
purity = np.sum(max_per_cluster) / np.sum(contingency_matrix)
```

For example in the case of the k-means (iterations=10) applied to the iris dataset we get the following confusion matrix:

$$\begin{pmatrix} 0 & 0 & 50 \\ 2 & 48 & 0 \\ 36 & 14 & 0 \end{pmatrix} \tag{2}$$

and we get a purity score of:

$$P_{km} = 0.893 \tag{3}$$

6.1.2 Davies-Bouldin index

The DBI (Davies-Bouldin index) is an internal metric (doesn't need the ground thruth) introduced by Daviv L. Davies and Donald W. Bouldin.

We first need to define the qth moment of the points in cluster i about the mean:

$$S_i = \left(\frac{1}{T_i} \sum_{j=1}^{T_i} |X_j - A_i|^q \right)^{\frac{1}{q}}$$

where A_i is the centroid of cluster i and T_i is the number of points in cluster i.

Clusters i and j are characterized by the Minkowski metric:

$$M_{ij} = \left(\sum_{k=1}^{N} |A_{ik} - A_{jk}|^p \right)^{\frac{1}{p}} \tag{4}$$

(for $p = 1$ M_{ij} is the Mahattan distance, for $p = 2$ M_{ij} is the Euclidian distance)

We can now define the similarity between clusters:

$$R_{ij} = \frac{S_i + S_j}{M_{ij}} \tag{5}$$

Of course it is ideal that M_{ij} (separation between two clusters) is as large as possible, and S_i as low as possible.

To define a metric R_{ij} we ask these properties:

- $R_{ij} \geq 0$
- $R_{ij} = R_{ji}$
- When $S_j \geq S_k$ and $M_{ij} = M_{ik}$ then $R_{ij} > R_{ik}$
- When $S_j = S_k$ and $M_{ij} \leq M_{ik}$ then $R_{ij} > R_{ik}$

a solution that satisfies these properties is the one defined before:

$$R_{ij} = \frac{S_i + S_j}{M_{ij}} \tag{6}$$

For each cluster we calculate the maximum of this value:

$$R_i = \max j \neq i R_{ij} \tag{7}$$

We can now define the Davies-Bouldin index for N clusters:

$$\overline{R} = \frac{1}{N} \sum_{i=1}^{N} R_i \tag{8}$$

```python
from sklearn.metrics import davies_bouldin_score
db_index=davies_bouldin_score(dataset, labels)
```

6.1.3 Infomax based metric $\hat{H}[s]$

We can interpret a clustering algorithm as a translation of the data into a message (in the language of possible labels).
As we know the information content of this message can be quantifies by the Shannon entropy, so we take as our metric:

$$\hat{H}[s] = -\sum_s \frac{K_s}{M} \log\left(\frac{K_s}{M}\right)$$

where K_s is the number of points in the cluster s and M is the total number of points in the dataset. If we take the principle of maximum information preservation as a natural and universal criterium for scoring different algorithms we have that if algorithm A_1 extracts more information than A_2 from the dataset ($\hat{H}_{A_1}[s] > \hat{H}_{A_2}[s]$), A_1 should be preferred.

6.2 K-means

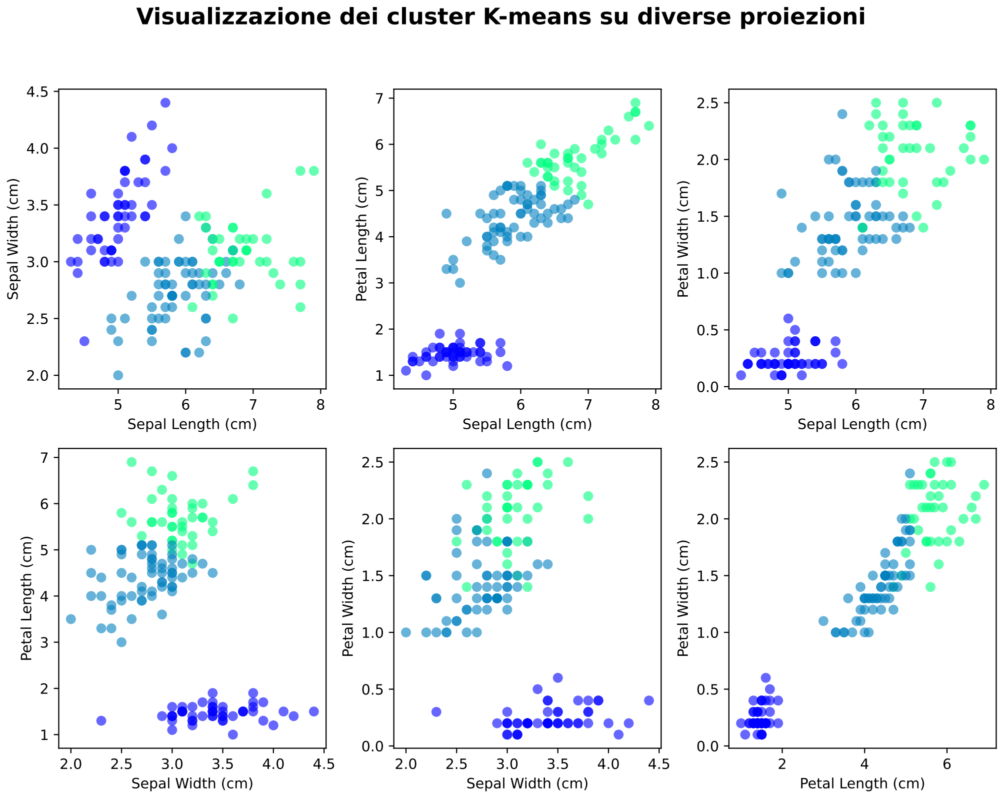

Figure 2: Visualization under different projections of the result of a k-means clustering on the Iris Dataset

Let's now study the convergence of the k-means algorithm under the purity score:

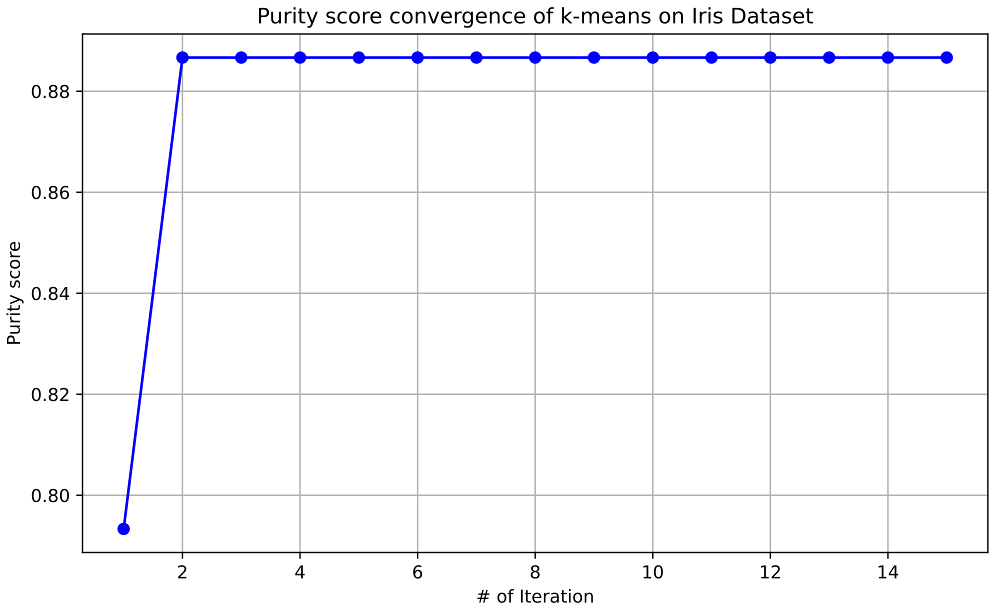

Figure 3: Example of the layout of a binary confusion matrix

It is also interesting that sometimes we get the following behaviour:

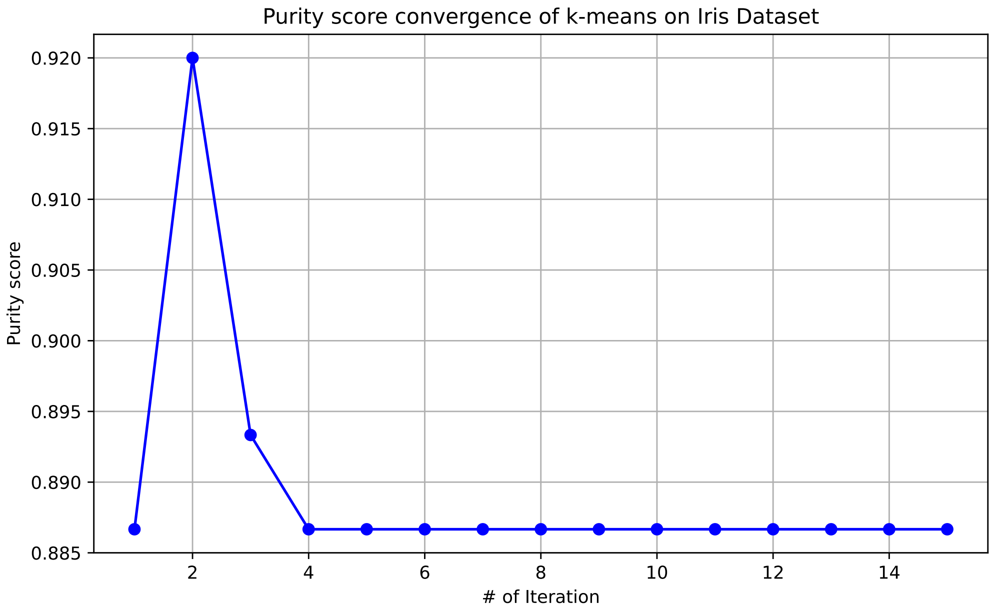

Figure 4

We can investigate how the algorithm works if we change the initial random centroids:

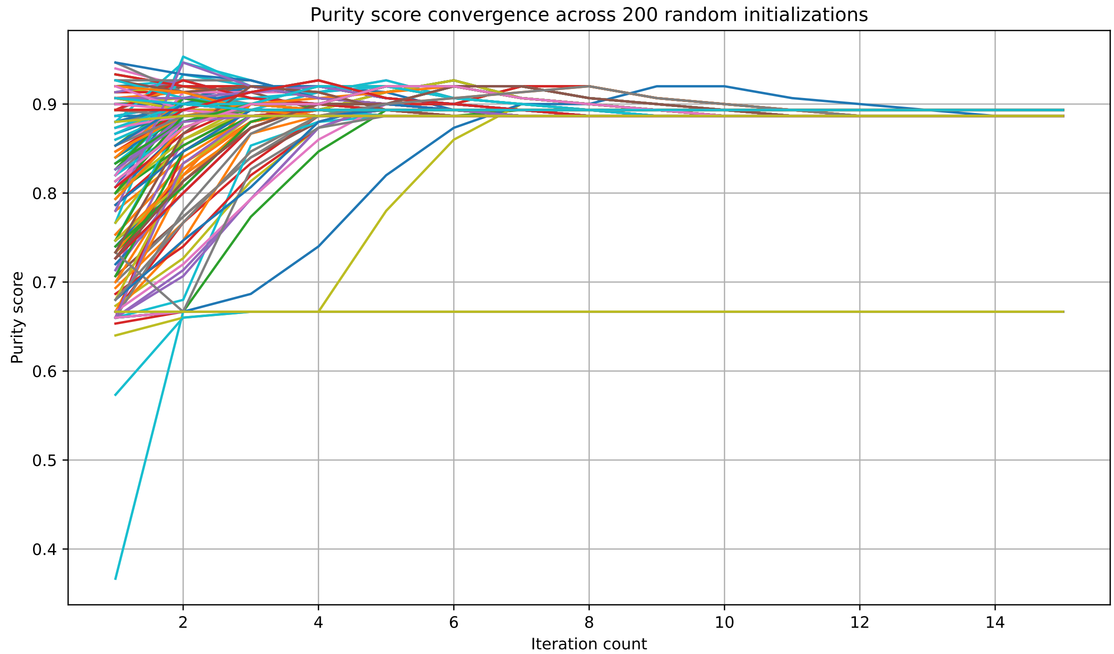

Figure 5: Purity score convergence across 200 random initializations

As we can see it seams that k-means is a non-ergodic algorithm: the convergence point depends on the initial conditions.

The same happens if we analize it with different metrics (D-B score, Infomax index):

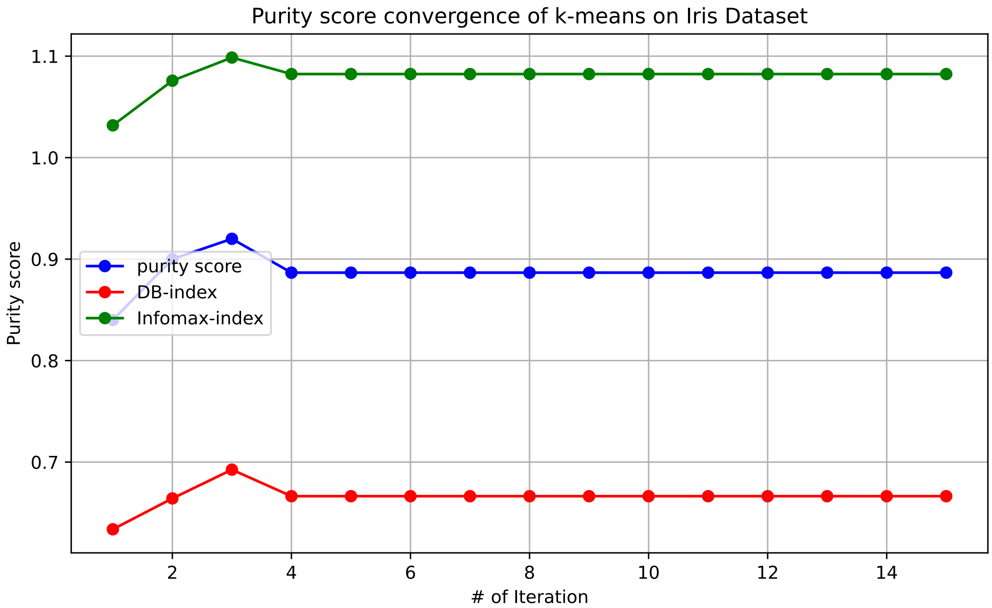

Figure 6: Comparison of different indexes against number of iteration of k-means algorithm

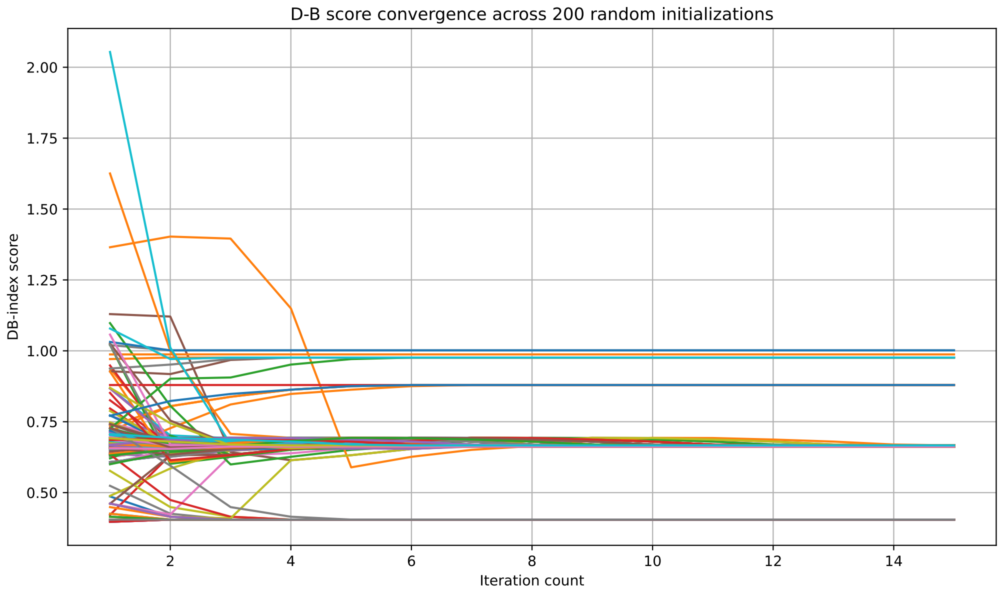

Figure 7: D-B score against 200 random initializations (k-means)

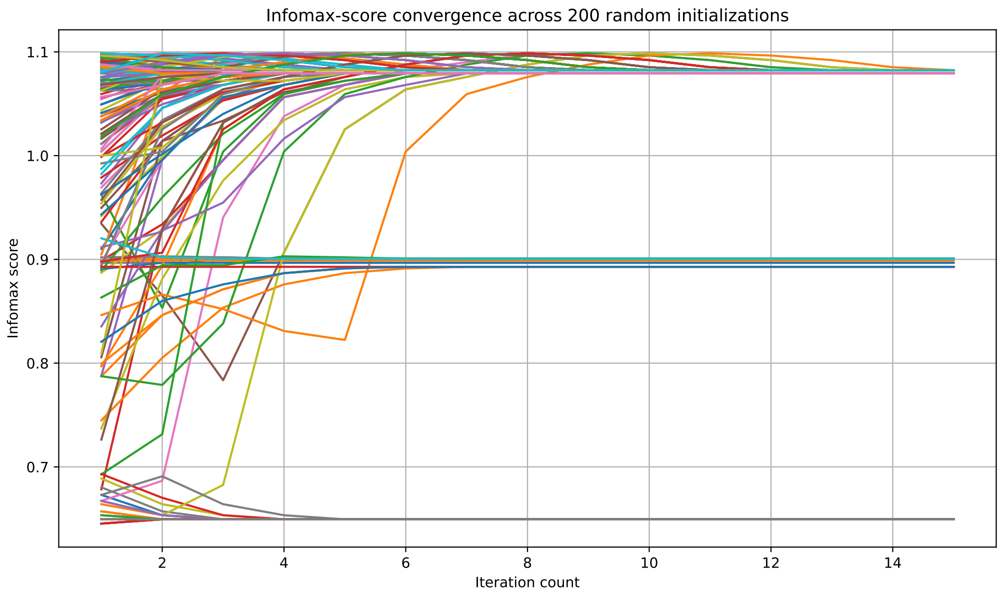

Figure 8: Infomax score against 200 random initializations (k-means)

6.3 Soft k-means

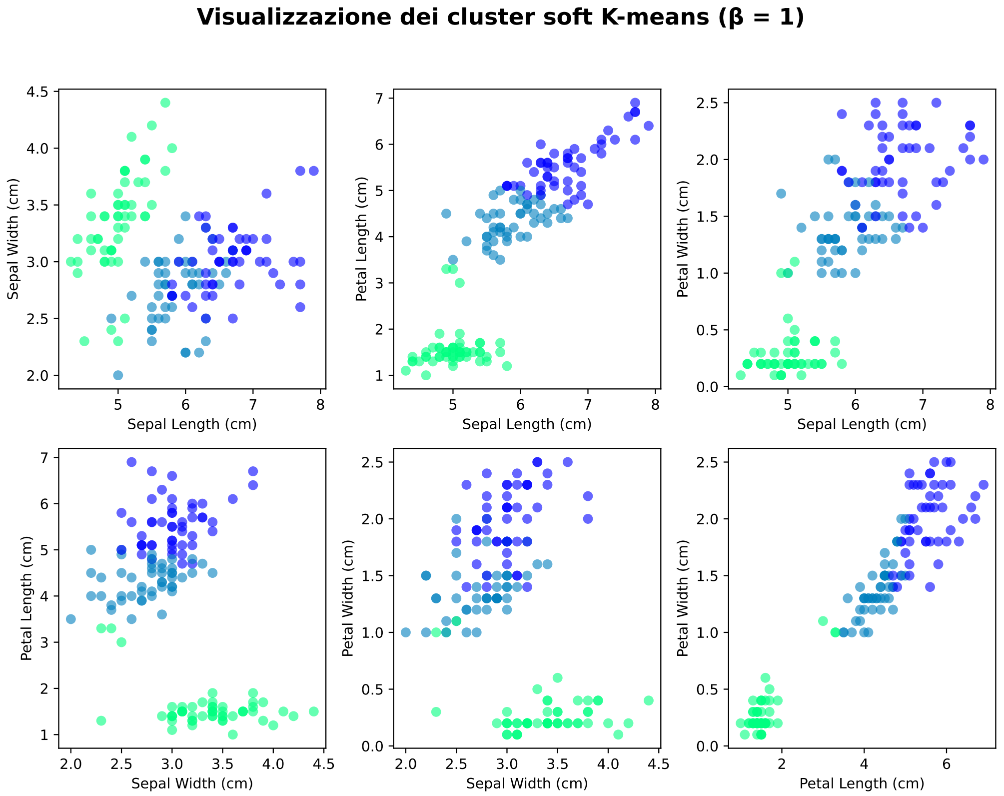

Figure 9: Visualization under different projections of the result of a soft k-means clustering on the Iris Dataset with $\beta = 1.0$

By analysing the soft k-means algorithm using the purity index we find that the convergence is much faster than the hard version:

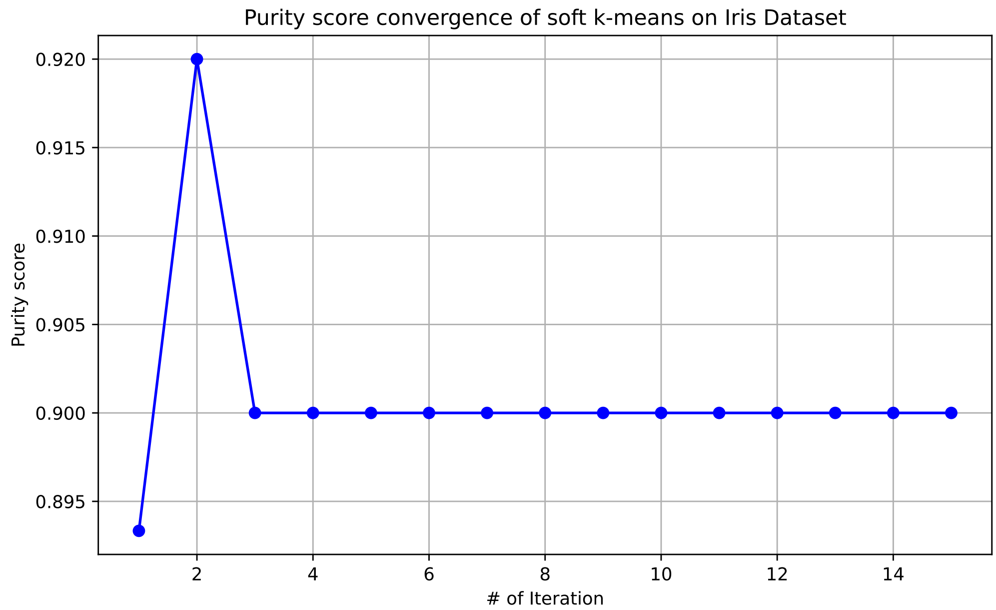

Figure 10: Purity index against number of iterations (soft k-means)

And by confronting the different metrics with a large number (500) of different initializations we find that the soft version seems to be ergodic (the convergence point is independent from initial conditions):

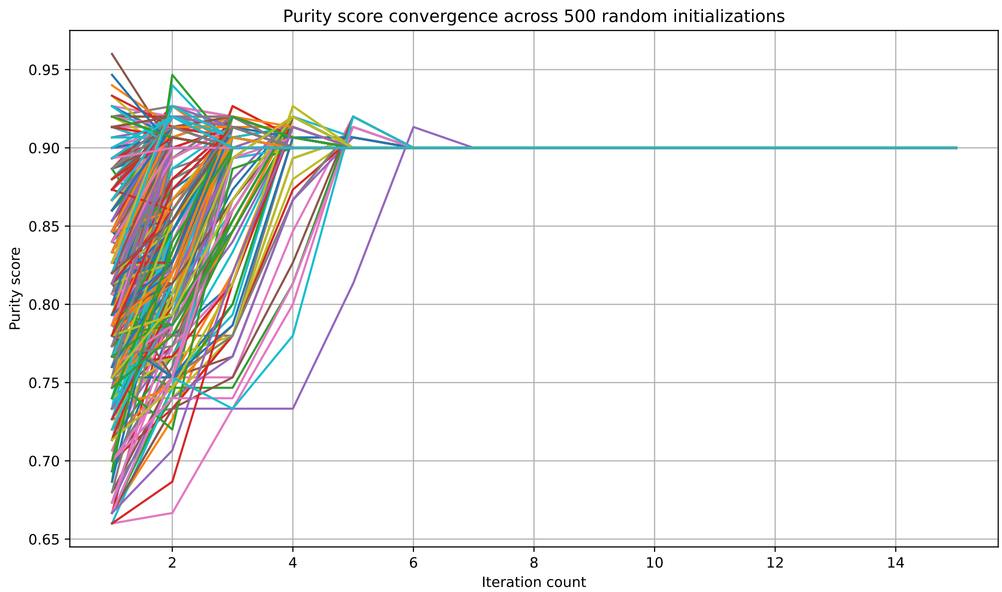

Figure 11: Purity index against 500 different initializations (soft k-means)

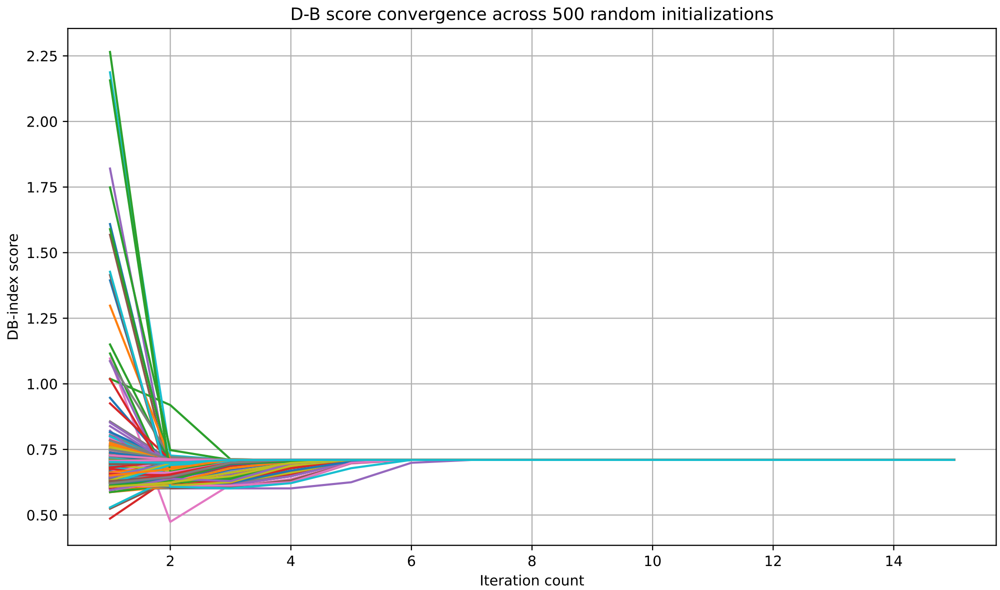

Figure 12: D-B score against 500 different initializations (soft k-means)

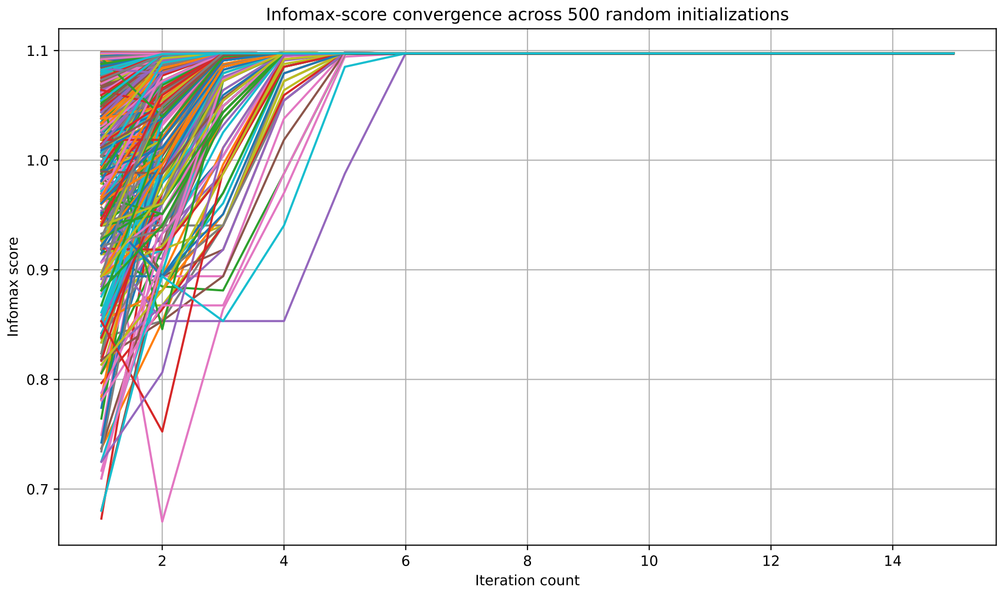

Figure 13: Infomax index against 500 different initializations (soft k-means)

But we cannot be sure of its ergodicity since we have done a limited number of trials (500) and the fact that the indexes of final clustering are the same doesn't necessarily mean that the final clustering labels are the same.

6.4 Gaussian Mixtures clustering

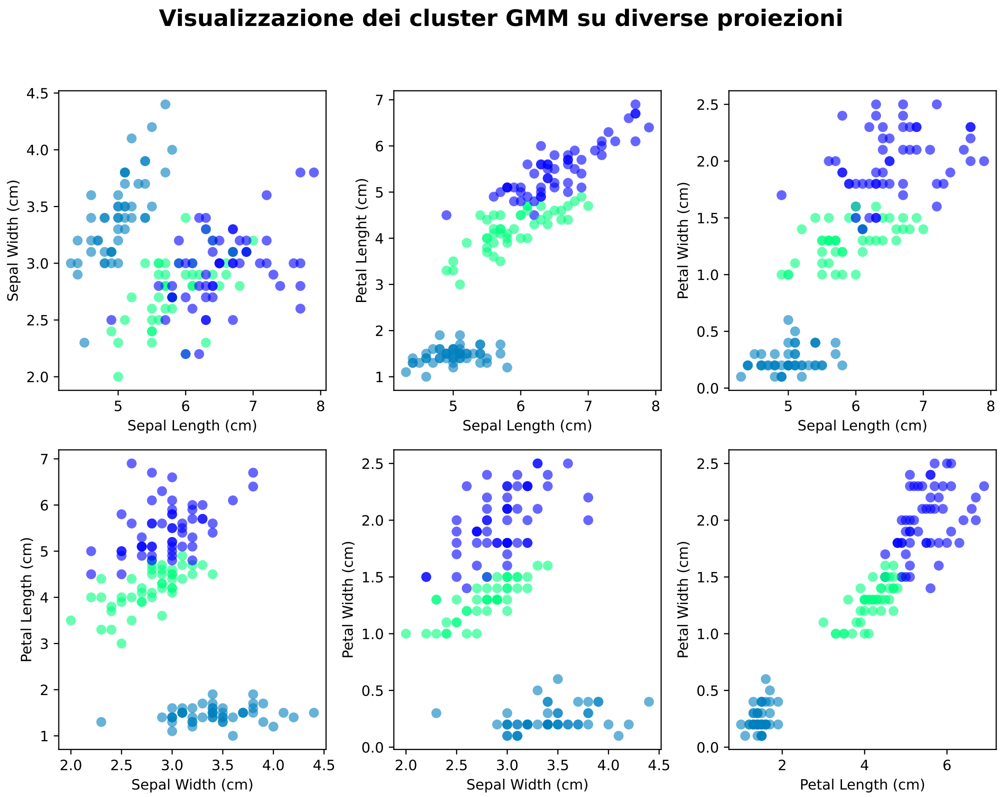

Figure 14: Visualization under different projections of the result of a GMM clustering on the Iris Dataset

From executing the algorithm we get:

- Purity score of Gaussian Mixtures: 0.967
- D-B score of Gaussian Mixtures: 0.749
- Infomax index of Gaussian Mixtures: 1.09

Based on the purity score this algorithm is much better than both the k-means and soft k-means. The other indexes are similar to the other algorithms.

6.5 CLL clustering

From executing the algorithm we get:

- Purity score of CLL: 0.887
- D-B score of CLL: 0.667
- Infomax index of CLL: 1.08

The lower purity score tells us that this algorithm doesn't respect the ground thruth as well as the others. But we have a D-B score much lower, and this tells us that the clusters we get with this algorithm are more separated and overlap less. The infomax index is similar, telling us that we are extrapolating pretty much the same quantity of information from the database.

7 Scikit-learn and other Python functions

7.1 Confusion Matrix

A confusion matrix, also known as error matrix is a specific table that allows visualization of the performance of an algorithm. Each row of the matric represents the instances in an actual class while each column represents the instances in a predicted class, therefore the diagonal of the matrix represents all instances that are correctly predicted. It is called confusion matrix because it lets you check easily whether the system is confusing two classes.

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Figure 15: Example of the layout of a binary confusion matrix

```
import numpy as np
from sklearn.metrics import confusion_matrix
matrix = confusion_matrix(y_true, y_pred)
```